

Unit 08: Control Structure

CONTENTS

Objectives

Introduction

8.1 Designing Structured Programs in C

8.2 Top Down Design

8.3 Type Conversion in C

8.4 Type Modifiers

8.5 Decision-making Control Statement

8.6 nested if statement

8.7 Looping

8.8 Jump and Break Statement

8.9 goto Statement

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Explain decision making in C
- Explain branching
- Describe if, if-else statement in C
- Explain switch statement
- Explain looping concept in C
- Describe do-while loop
- Describe goto statement

Introduction

To write a realistic programme, don't think of it as a collection of statements put in a specific order. It takes a lot more than that. Take a look at a real-life example. Life does not always go as planned:

1. There are some situations when you have to take decisions like whether to purchase this book or not.
2. There are also some situations where you have to perform the particular action again and again like for better understanding read this unit 5 times continuously.

In the same way, it's rare that we write a computer programme that doesn't run into similar issues. Depending on the circumstances, most programmes require a statement or combination of statements to be executed numerous times or not at all.

The C programming language offers a number of programme control statements that allow you to influence the order in which your programmes are executed. This unit looks at different iteration-based programme control statements and how to use them in a programme. The break and continue jump statements in C are also discussed in this section.

8.1 Designing Structured Programs in C

A C program is divided into different sections. There are six main sections to a basic c program.

- Documentation
- Link
- Definition
- Global Declarations
- Main functions
- Subprograms

Documentation Section

The documentation area of the programme is where the programmer provides information about the programme. He normally offers the program's name, author information, and other facts such as coding time and description. It provides a high-level overview of the code to anyone reading it.



Example:

```
**  
  
* File Name: Helloworld.c  
* Author: Name  
* date: DD/MM/YYYY  
* description: a program to display hello world  
*           no input needed  
*/
```

Link Section

This part of the code is used to declare all the header files that will be used in the program. This leads to the compiler being told to link the header files to the system libraries.



Example: #include<stdio.h>

Definition Section

In this section, we define different constants. The keyword define is used in this part.



Example: #define PI=3.14

Global Declaration Section

The declaration of global variables takes place in this section of the code. This section declares all of the global variables that are used. In this section of the code, the user-defined functions are also declared.



Example: float area(float r);

int a=5;

Main Function Section

In all C programmes, the principal role is necessary. Each major role is divided into two sections. This has two parts: a declaration section and an execution section. The declaration section contains all of the variables. Curly brackets are used to begin the execution and curly close brackets are used to finish it. Both the declaration and execution parts are contained within the curly braces.

```
int main(void)
{
    int a=500;
    printf(" %d", a);
    return 0;
}
```

Sub Program Section

This section of the software defines all user-defined functions.

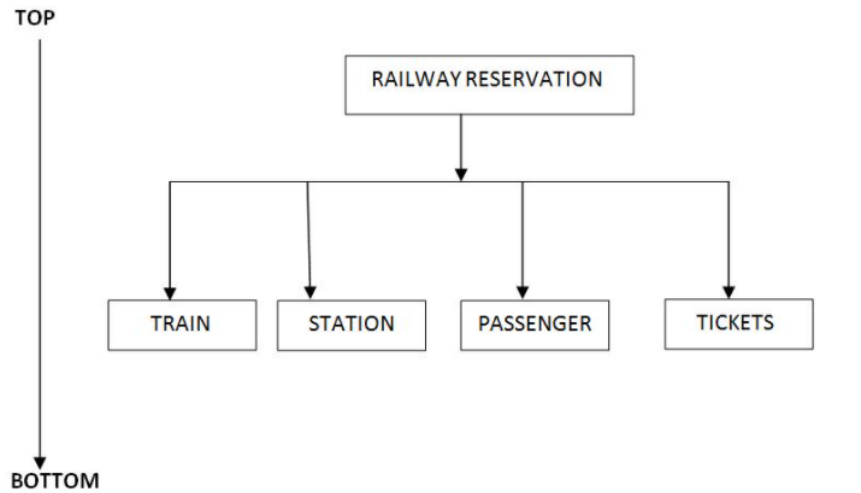
```
int add(int a, int b)
{
    return a+b;
}
```

8.2 Top Down Design

In this strategy, a huge project is broken down into smaller projects known as modules. The C programming language facilitates this technique of project creation. It's often a good idea to break down a solution into modules in a hierarchical approach. The main aim of a top-down method is to break down the problem into tasks, which are then broken down into smaller sub-tasks, and so on. The primary module is built initially, followed by the next level modules in this manner. This procedure is done until all of the modules are complete.

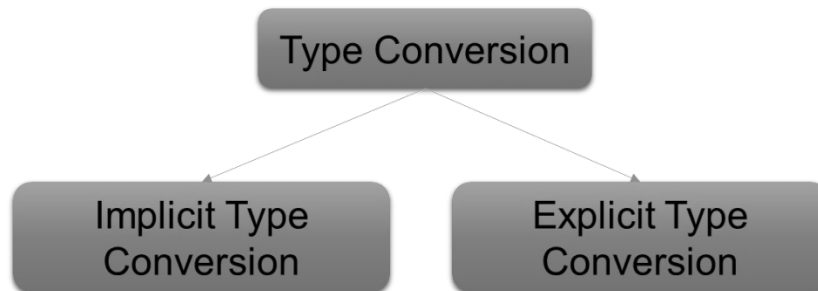
Advantages of Top-Down Approach:

1. In this approach, first, we develop and test most important module.
2. This approach is easy to see the progress of the project by developer or customer.
3. Using this approach, we can utilize computer resources in a proper manner according to the project.
4. Testing and debugging is easier and efficient.
5. In this approach, project implementation is smoother and shorter.
6. This approach is good for detecting and correcting time delays.



8.3 Type Conversion in C

Converting one type of data type to other to perform some operations. Conversion is done on those data types where conversion is possible.



Implicit Type Conversion

This type of conversion is done by compiler whenever necessary automatically without any commands by user. This conversion is performed by compiler when a particular expression contains more than one data type.



Example: `int a = 10;`

`double b = 50.5, sum;`

`sum = a + b;`



Lab Exercise:

```
#include<stdio.h>
```

```
int main(){
```

```
int a=10;
```

```
double b=9.34, sum;
```

```
sum=a+b;
```

```
printf("%lf", sum);
```

```
return 0;
```

```
}
```

Explicit Type Conversion

Without the use of compiler when one data type is changed to another data type, user explicitly defines within the program the data type of the operands in the expression.



Example:

```
double a = 4.5,b=3.5;
//explicitly defined by user
int sum=(int)a+(int)b;
```

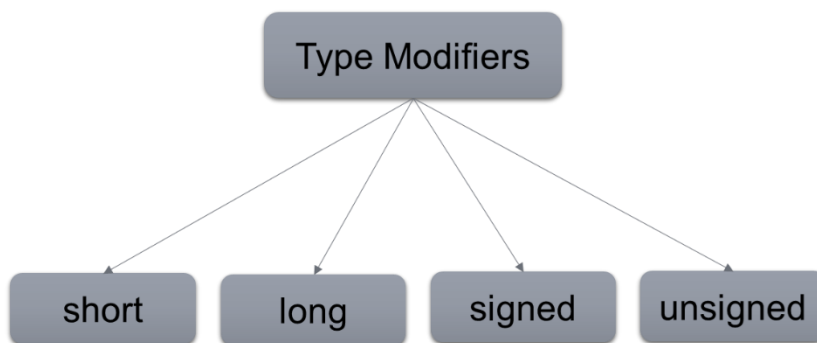


Lab Exercise:

```
#include<stdio.h>
int main(){
double a = 10.5,b=5.5;
//explicitly defined by user
int sum=(int)a+(int)b;
printf("%d",sum);
return 0;
}
```

8.4 Type Modifiers

In c language Data Type Modifiers are keywords used to change the current properties of data type.



Long

This can be used to increased size of the current data.it is applied on int or double data type.



Example: long double a;



Lab Exercise:

```
#include<stdio.h>
```

```
int main(){
long double a ;
printf("%d",sizeof(a));
return 0;
}
```

Short

In general int data type occupies different memory spaces for a different operating system; to allocate fixed memory space short keyword can be used.



Example: short int a;



Lab Exercise:

```
#include<stdio.h>
int main(){
short int a ;
printf("%d",sizeof(a));
return 0;
}
```

Unsigned

This keyword can be used to make the accepting values of a data type is positive data type.



Example: unsigned int a=10; //right
 unsigned int a= -50; //wrong



Lab Exercise:

```
#include<stdio.h>
int main(){
unsigned int a=-10 ;
printf("%u", a);
return 0;
}
```

Signed

This keyword accepts both positive and negative value.



Example: unsigned int a=10; //right
 unsigned int a= -50; //right



Lab Exercise:

```
#include<stdio.h>

int main(){
    signed int a=-10 ;
    printf("%d", a);
    return 0;
}
```

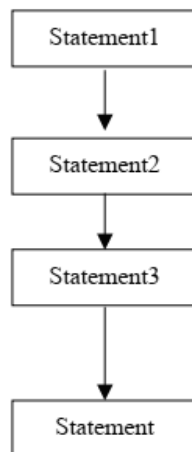
8.5 Decision-making Control Statement

Control flow statement or programme control statement is a statement that allows us to control the flow of a program's execution. Sequentially, selectively, or iteratively, programme statements can be performed. The C programming language includes constructs for sequence, selection, and iteration. The program's flow is explained by a mix of one or more of the following structures.

1. Sequence
2. Selection
3. Iteration

Sequence

In the sequence construct, as the name implies, statements are executed sequentially i.e. one after the other. In this, neither the statement are repeated nor in the order of execution changed as shown in Figure.



You probably noticed in the previous unit that a C programme is executed top down, that is, execution begins with the beginning of the main() function and continues, statement by statement, until the end of the main (). In a C programme, the following programme illustrates the sequential execution of statements.



Lab Exercise:

```
#include<stdio.h>
```

```

main(){
printf("\n First statement");
printf("\n Second statement");
printf("\n Third statement");
printf("\n Second last statement");
printf("\n Last statement");
}

```

Output : -

```

First statement
Second statement
Third statement
Second last statement
Last statement
Process returned 0 (0x0)   execution time : 0.028 s
Press any key to continue.

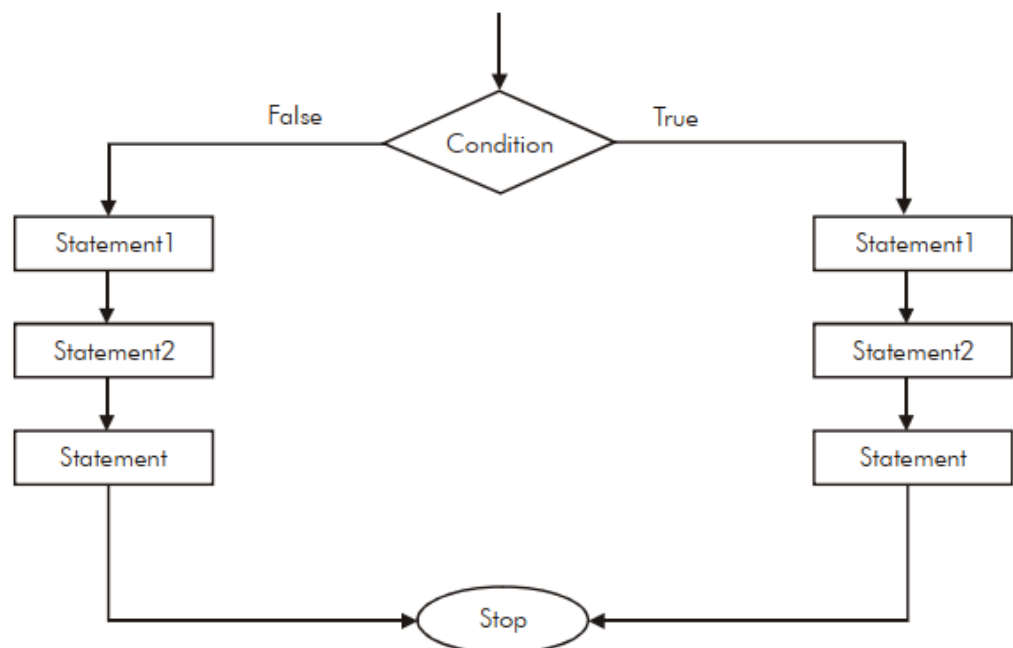
```



Notes: Other complex statement may be used in a sequence to demonstrate the concept.

Selection

Only appropriate statements are often performed instead of all of them, depending on the input and the circumstance. A condition test is used to execute statements in the selection construct. You advise the computer to do one course of action if the test returns true; otherwise, you advise the programme to take an other course of action. The selection construct contains two or more sets of statements, but only one of them is executed, as shown in the Figure.



The selection construct can be implemented by means of the if construct. The if construct makes use of relational and logical operators for decision making, as shown in the pseudo code given below:

**Lab Exercise:**

```
#include<stdio.h>

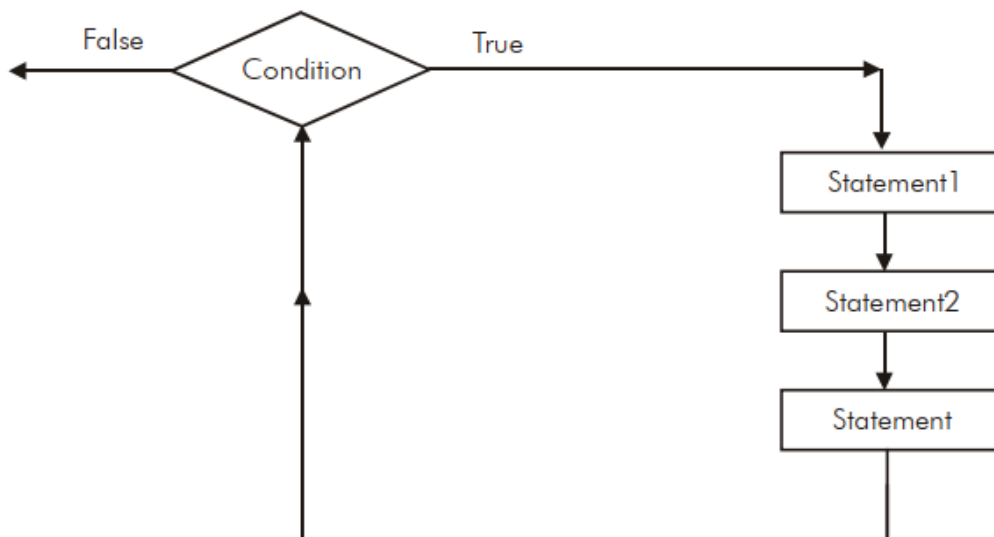
main(){
int marks=50;
if(marks>40)
    printf("Pass");
else
    printf("Fail");
}
```

Output: -

```
Pass
Process returned 0 (0x0)   execution time : 0.010 s
Press any key to continue.
```

Iteration

The iteration constructs are an efficient method of handling a series of statements that must be repeated a variable number of times. Sometimes the required number of repetitions is known in advance and sometimes the statements repeat over and over until certain specified conditions are met, as shown in the Figure.



The iteration construct is also called as loop. The statements that are to be executed are called as body of the loop and the condition on which a loop terminates is called as exit condition as demonstrated by the program given below:

**Lab Exercise:**

```
#include<stdio.h>

main(){
    int i=0;
    while(i<=20){
        printf("\n %d ",i);
        i++;
    }
}
```

Output: -

```
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
Process returned 0 (0x0)   execution time : 0.070 s
Press any key to continue.
```

Simple if Statement

In its basic form, the if statement evaluates a test condition (i.e., nothing but an expression) and direct program execution depending on the result of that evaluation. The general form of a simple if statement is as shown below:

```
if (expression)
    statement;
```

Where a statement may consist of a single statement, a compound statement or nothing as an empty statement. The expression also referred so as test condition must be enclosed in parentheses, which causes the expression to be evaluated first. If it evaluate to true (i.e., a non-zero value), then the statement associated with it will be executed otherwise ignored and the control will pass to the next statement.



Example: Consider the following statement:

```
:
if (marks > 9)
printf ("\n Pass");
:
:
```

The above code fragment will printf "Pass" on the monitor if the value of marks is greater than 9. If the value of marks is not greater than 9, the control simply ignores this statement and will pass to the next statement. The following program shows the use of simple if as it accepts the marks of a student and prints his/her result.



Lab Exercise:

```
#include<stdio.h>

void main
{
int marks;
printf( "\n enter your marks");
scanf("%d", marks);
if (marks>9) /* if construct with test condition */
printf( " \n Pass"); /* statement associated with if */
printf(" Thank you"); //next statement
}
```

output: run 1-> run 2->

Enter your marks : 77 ↵ Enter your marks : 9 ↵

Pass Thank you

Thank you



Notes: "Pass" has been displayed only if the expression evaluated to true otherwise if it evaluated to false, the control ignores the associated statement and executes the next statement i.e., "Thank you".

As mentioned earlier, an if statement can control the execution of multiple statements, called as compound statement or a block. Where a block is a group of two or more statements enclosed in braces. So if these multiple statements are to be executed then they must be placed within a pair of braces, as illustrated by the following program.



Lab Exercise:

Program:

```
# include<stdio.h>

void main()
{
int marks;
```

```
printf ("\n Enter your marks: ");
scanf (" %d ", &marks) ;
if (marks > 39)
{
printf("\n Pass");
printf (" \n Congratulation ...");
}
if (marks <40)
{
printf ("\n Fail");
printf ("\n sorry. Good luck next time ...");
}
printf ("\n Thank you");
}
```

Output: run 1-> run 2->

Enter your marks: 77⁴ Enter your marks : 20⁴

Pass Fail

Congratulations... Sorry. Good luck next time....

Thank you Thank You



Notes: Default an if construct when evaluates to true executes only the first statement associated with it. If multiple statements are not enclosed with in parentheses, results may be unexpected.



Task: Write a program if candidate got more than 50% in year exam screen show "Pass" otherwise "Fail".

if-else statement

When the specified expression evaluates to true, the basic if statement runs a single statement or a collection of statements, as you may have noticed in the previous section (i.e. non-zero value). When the expression evaluates to false (i.e. a zero value), it does nothing and proceeds to the next step in the programme.

If you want a statement or a set of statements to be performed, however. Only when an expression evaluates to false can it be mentioned in the else part, as demonstrated below in the if else statement's standard structure.

```
if (expression)
statement1;
else
statement 2;
```

If expression evaluates to true, statement 1 is executed. If expression evaluates to false, statement 2 is executed, but never both. Both statement 1 and statement 2, as mentioned earlier, may be single statement, a compound statement, or an empty statement.

Actually, the simple if statement described in previous section is a simplification of its parent statement i.e. if else statement, where the else section is optional. Without it, however, an if-else construct looks like a simple if construct.

if-else construct is particularly useful when you have the statements to be executed in both the cases i.e. when the expression evaluates to true or false.



Example: Consider the following statements:

```
:
if (marks >= 40)
printf ("\n Pass");
else
printf ("\n Fail");
:
```

The code segment will display "Pass" on the monitor if the value of marks is greater than or equals to 40. If the marks are less than 40 (obviously the else case), then the statement in the else section will be executed and will printf "Fail" on the monitor. Let's write the Program by using an if-else construct.



Lab Exercise:

```
#include<stdio.h>

void main ()
{
int marks;
printf ("\n Enter your marks:");
scanf ("% d", & marks);
if (marks >=40)
{
printf ("\n Pass");
printf ("\n congratulations....");
}
Else
{
printf ("\n Fail");
printf ("\n Sorry. Good luck next time ...");
}}
}
```

Output: run1 → run2 →

Enter your marks: 77 → Enter your marks : 30 →

Pass Fail

Congratulations.... Sorry. Good luck next time...



Notes: else section required their own pair of braces as more than one statement is to be executed when the expression evaluates to false.

8.6 nested if statement

The executions of the inner if depends upon its location in the outer construct and upon the value of expression of the outer construct. For instance, consider the following:

```
:
if (expression 1)
{
    if (expression 2)
        statement 1;
    else
        statement 2;
} :
```

In the above segment of code, the inner if executes only if the expression1 evaluates to true. The other possible combination of nested if may take one of the following form:

```
· if (expression1)
{
    if (expression2)
        statement1;
}
· if (expression1)
{
    if (expression 2)
        statement1;
    else
        statement 2;
}
· if(expression1)
{
    statement1;
}
else
{
    if(exp2)
        statement 2;
}
· if (exp 1)
{
```

```
statement1;
}
else
{
if (exp2)
statement 2;
else
statement3;
}
if(exp 1)
{
if (exp 2)
statement1;
else
statement 2;
}
else
{
if (exp3)
statement3;
else
statement4;
}
if (exp1)
{
if (exp2)
statement1;
}
else
{
if (exp3)
statement 2;
}
}
```

Let's write a couple of programs to explore the various combination of nested if. The following program not only print the request after accepting marks from the students but also print his/her grade.

**Lab Exercise:****Program**

```
#include<stdio.h>
void main ()
```

```
{
int marks;
printf ("\n Enter your marks:");
scanf ("% d",&marks");
if (marks >=40)
{
printf ("\n Pass");
if (marks > = 80)
printf (" with distinction");
}
else
printf ("\n Fail");
}
```

Output: run 1 ->

Enter your marks : 77↵

Pass

Run 2->

Enter your marks : 88↵

Pass with distinction

Run 3->

Enter your marks : 38↵

Fail



Notes: The execution of the inner if only be there if first expression evaluates to true and the message "with distinction" will printf only if both the expressions evaluates to true.

Nested else-if statement

Imagine a situation where you have to test number of conditions to get the desired results. These types of particular situations requires nestedness of if-else statements up to a deeper level and it may looks like as:

```
if (expression 1)
statement 1;
else
if (expression 2)
statement 2;
else
if(expression 3)
statement 3;
else
:
```


statement n;

The following program demonstrates the use of nested if-else statement up to a deeper level. This program will accept the marks of a student and will display the grade accordingly.



Lab Exercise:

Program:

```
# include<stdio.h>

void main ()
{
    int marks;
    char grade;
    printf ("\n Enter your marks:");
    scanf ("% d", & marks");
    if (marks > = 90)
        grade = 'O';
    else
        if (marks > = 80)
            grade = 'D';
        else
            if( marks > =75)
                grade = 'M';
            else
                if (marks > = 60)
                    grade = 'I';
                else
                    if (marks > =50)
                        grade = 'II';
                    else
                        if (marks > 40)
                            grade = 'III';
                        else
                            grade = 'F';
    printf( "\n Your grade is : % c", grade),
}
```

Output: run 1 →

Enter your marks: 77 ↵

Your grade is: M

Run 2 →

Enter your marks: 39 ↵

Your grade is: F



Notes: This whole section of code is actually one statement that is comprised of six hierarchically nested if else constructs, so there is no need to put them in the braces. At any time during the general top to bottom execution of these expressions, if an expression evaluates to true, then the associated statement will be executed and control flow will pass to the statement immediately following the entire nested chain.

Switch statement

The switch statement is another useful C feature for dealing with scenarios where numerous decisions must be made based on an expression with multiple values. The switch is a multiple branch statement that compares the value of an expression to a list of case values and, if a match is found, executes the statement associated with that case. A switch-case statement can take the following general form:

```
switch (expression)
{
    case value1: statement1;
    case value2: statement2;
    case value3: statement3;
    :
    case valuen: statementn;
    [default: statement x ;]
}
```

statement;

Where switch is a keyword and the expression is any expression that evaluates to an integer value, may be of type int, or char, or long. The case is a keyword followed by value 1, value 2, value n, where value 1, value 2, .. value n may be an integer or character constant, normally referred to as case labels. And the statement1, statement2, .. statementn may be single statement or set of statements, or may be an empty statement.

The switch statement evaluates the expression first and then compare the return value against the values value1, value2,.. valuen, and then one of the following happens:

1. If a case is found whose value matches with the value of the expression then the statement associated with that case is executed.
2. If no match is found then the statement followed by the keyword default is executed.
3. If no match is found and there is no default label as it is an optional case, then no action takes place and control passes to the statement next which is a statement immediately followed the switch statements closing braces.

Consider the following program, which gives you an example of using the switch statement. This program will receive a number between 1 to 5 and will display it's English counterpart.



Lab Exercise:

Program:

```
#include<stdio.h>

void main ()
{
    int num;
```

```
printf ("\n Enter any number between 1 to 5 : ");
scanf ( " % d", &num );
switch (num)
{
case 1 : printf ("\n One");
case 2 : printf ("\n Two");
case 3 : printf ("\n Three");
case 4 : printf ("\n Four");
case 5 : printf ("\n Five");
default : printf ("\n Wrong input");
}
printf ("\n Thank You");
}
```

Output: run 1 → run 2 →

Enter any number between 1 to 5: 2↵ Enter any number between

1 to 5: 4↵

Two Four

Three Five

Four Wrong input

Five Thank you

Wrong input

Thank you

Run 3->

Enter any number between 1 to 5 = 9↵

Wrong input

Thank you



Notes: There is no need to put braces with the individual case labels as they each contains single statement, although a pair is required to group the entire case section.

8.7 Looping

Iteration statements are also known as loops or looping statements because the program execution typically loops through the statement more than once. In this category, C provide the following statement or you call loops.

1. for loop
2. while loop
3. do-while loop

Looping must not continue indefinitely as an analogy to real life you would not like to crack the same joke again and again, so a mechanism is required to break out the loop and to allow the executives of the next set of statements.

As a result, a general structure for implementing a loop expression has been designed. Which can be better understood by grasping the following elements/parts/components of a loop that regulates the amount of repetitions:

1. **Initial Expression(s):** Initial expression(s) is usually an assignment expression(s) which initializes the control variable(s) of a loop, as they must be initialized before entering in a loop. The initial expression(s) is executed only once, in the beginning of the loop. But if this expression(s) occurs in the loop body, control variable(s) would be reassigned to initial values with every loop pass, and the condition expression would never fail.
2. **Condition Expression:** Conditional expression is typically a relational expression that is set up to terminate the execution of a loop. If the condition expression evaluates, to true i.e. 1, the loop body gets executed, otherwise the loop is terminated. A condition expression may be evaluated before entering in to a loop or before exiting from the loop called as entry-controlled loop and exit controlled loop respectively. In C, the for loop and while loop are entry-controlled loops whereas do while loop is exit-controlled loop.
3. **Update Expression(s):** The update expression(s) is essentially an increment expression or decrement expression that changes the value(s) of loop variable(s), so that they could come to the boundary values. The update expression(s) normally execute at the end of the loop body. It may appear in the body of loop as it is updating expressions that assign the variable a new updated value every time the loop passes.
4. **The Loop Body:** The loop body consists of statement(s) that is supposed to be executed again and again as long as the condition expression evaluator to true i.e. 1. In an entry-controlled loop, the condition expression evaluated first and if it evaluates to true, the loop-body is executed and if it evaluate to false, the loop-body is terminated. Whereas, in exit controlled loop, the loop body executed first and then the condition expression are evaluated. It is evaluate to false i.e. 0, the loop is terminated, otherwise repeated.

The above mentioned components are the essential component of a statement to be called as a loop statement. Messing any of them may change the basic meaning of a perfect loop. The for, while and do-while statements of C, comprises of all these essential components, hence referred to as loop statements.

for loop

The for loop in C is the simplest, fixed and entry controlled loop. It is simplest as the structure of for loop is divided into two segments i.e. control statement and the body of the loop. All its loop control elements are placed together in the control statement where as body of the loop consists of statements to be executed repeatedly.

It is fixed as number of repetitions is known in advance and can be useful in a situation when you want to do something a fixed number of times. It is an entry controlled loop as the control statement placed before the loop body i.e. condition expression will be evaluated first. The general form of the for loop is:

`for(initial expression(s) ; condition expression ; update expression(s)) loop-body;`



Example: Consider the following statement:

```
for ( i =1 ; i <= 10; ++i)
```

```
printf("\n Hello World!");
```

where i is an integer variable declared already.

i=1; is an initial expression.

i < = 10; is a conditional expression.

++i; is an update expression.

And the statement

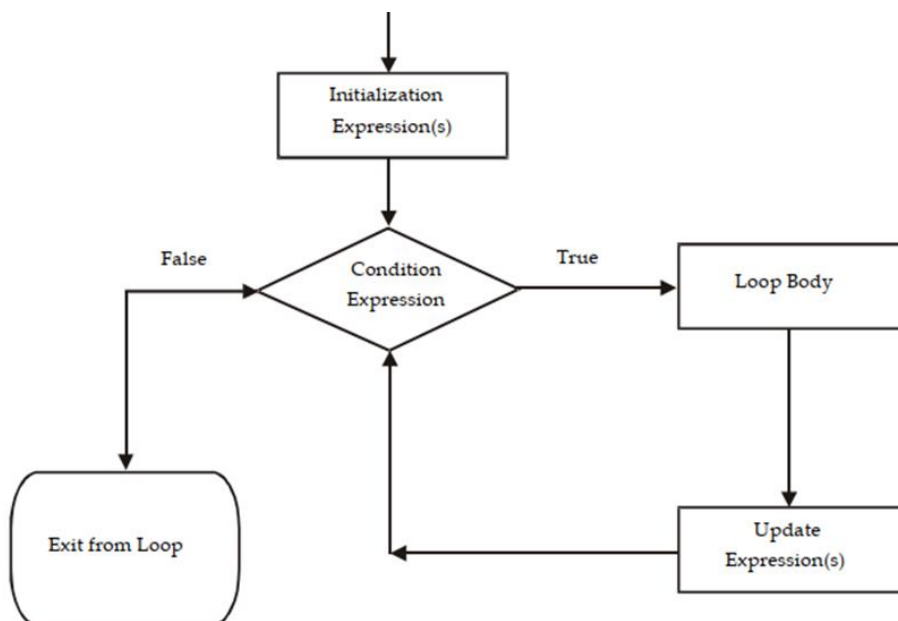
```
printf("\n Hello World!");
```

is the body of the loop.

When the above statement is encountered during program execution, the following events occur:

1. Initial expression is evaluated first and i will be assigned an initial value 1 i.e. $i = 1$.
2. Then the condition expression is evaluated i.e. $i \leq 10$ and the result will be true as $1 \leq 10$ is true.
3. Since the condition expression is true, the statement in the loop body is executed i.e. `printf("\n Hello World!");` which prints the message Hello World! on the screen.
4. After the execution of the loop body, the update expression i.e. $++i$ is executed which increment the value of i by 1. In this way after the first execution of the loop the value of i becomes 2 as initially it was 1.
5. After the execution of the update expression the condition expression is again evaluated. If it evaluates to true the sequence is repeated from step no. 3, otherwise the loop terminates.

Also note that the loop body never executes if condition expression is evaluated to false in its first execution. Figure shows the operation of a for loop.



Lab Exercise:

Program :

```
#include<stdio.h>
```

```
int main(){
```

```
int i,n;
```

```
printf("Enter Number");
```

```
scanf("%d",&n);
```

```
for(i=1;i<=10;i++)
```

```
{
```

```

printf("%d*%d=%d\n",i,n,i*n);

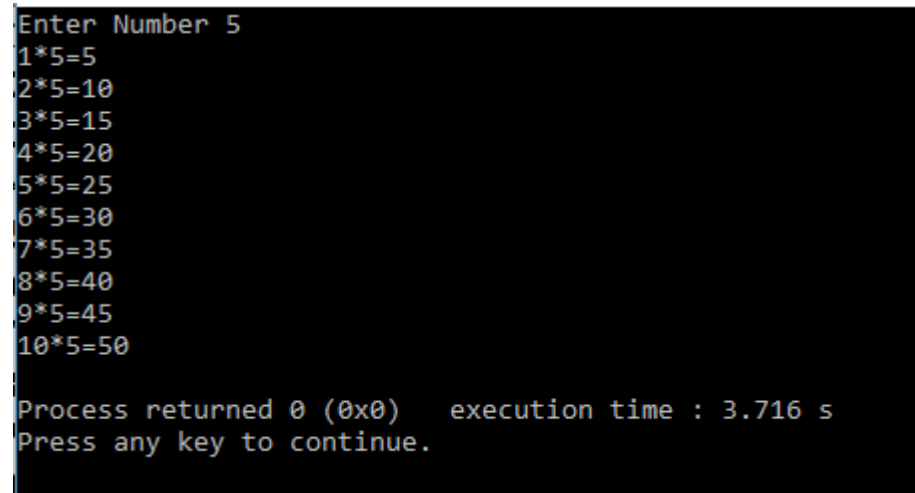
}

return 0;

}

```

Output : -



```

Enter Number 5
1*5=5
2*5=10
3*5=15
4*5=20
5*5=25
6*5=30
7*5=35
8*5=40
9*5=45
10*5=50

Process returned 0 (0x0)   execution time : 3.716 s
Press any key to continue.

```

While loop

The while loop, the second type of loop, is an entry controlled loop because it tests the conditions first and only the control enters the loop body if the condition is true.

When the loop's iterations are complete, the control returns to the while statement, which repeats the condition test. If the condition is false the first time, the loop does not iterate and control is passed to the statement after the loop statement. Because we don't know the precise amount of iterations, it's a kind of variable loop. The statements keep repeating themselves until specific conditions are met.

The while loop has the following form:

```

while (condition expression)
loop body;

```

Where the loop-body may contains a single statement, a compound statement or an empty statement. The while loop iterates the loop body as long as the specified condition expression evaluates to true.

The while loop doesn't explicitly contains the initialization expression and update expressions of the loop. These two expressions are normally provided by the programmers as the initialization expression(s) should be placed before the loop begins and updation expression(s) should be inside the loop body. By using all these expressions the general farm of while loop may looks like as:

```

:
initialization expression(s);
while (conditional expression)
{
:

: Loop Body

```

updatation expression;

}



Example:

Consider the following segment of code:

```
i = 1 ;
while ( i < = 10)
{
printf("\n Hello World!");
++ i;
}
```

where i is an integer variable declared already

i = 1; is an initial expression

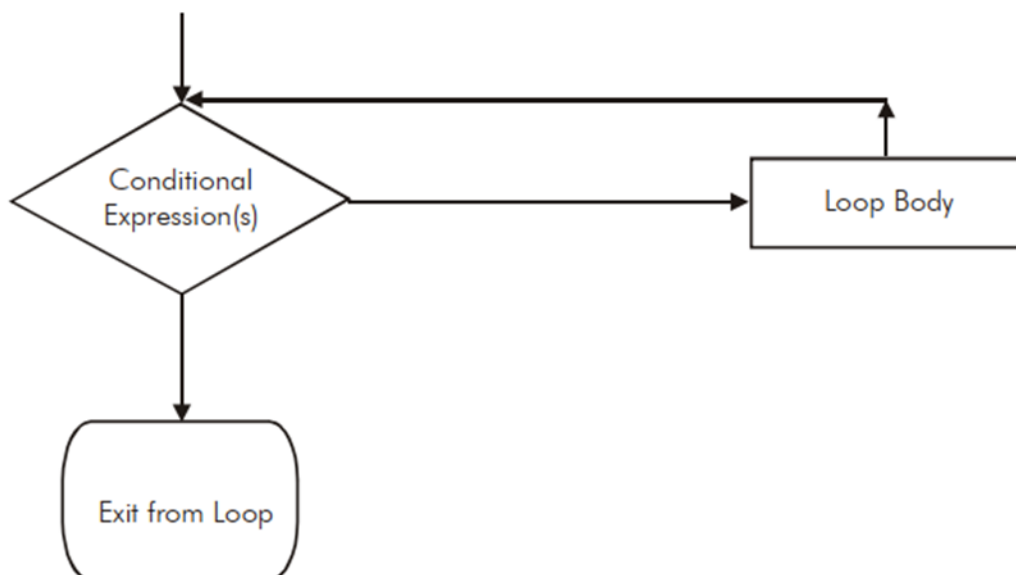
i < = 10; is a conditional expression

++i ; is an update expression.

And the statements between the { and } forms the body of the loop. But the braces can be discarded, if there is only one statement in the loop body

When the program execution readers a while statement, the following events occur:

1. First of all the conditional expression is evaluated i.e. $i < 10$.
2. The conditional expression is evaluated to true as i was 1 initially and $1 < 10$ is true. But if it evaluate to false, the loop will be terminated and the control moves to the first statement following loop body.
3. Since the condition expression is true, the loop body will be executed i.e. the printf statement and the updatation expression.
4. With the closing braces (}), it is assumed that the loop is finished and the control moves back to the while statement, which repeats the test again and proceeds accordingly.



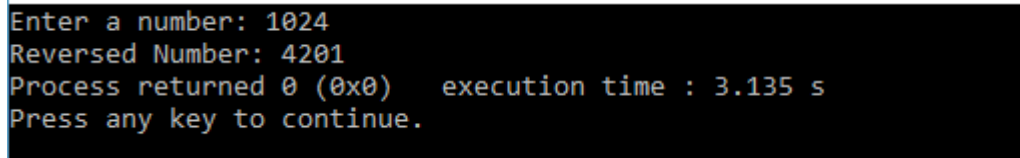
**Lab Exercise:**

Program : -

```
#include<stdio.h>

int main()
{
    int n, reverse=0, rem;
    printf("Enter a number: ");
    scanf("%d", &n);
    while(n!=0)
    {
        rem=n%10;
        reverse=reverse*10+rem;
        n/=10;
    }
    printf("Reversed Number: %d",reverse);
    return 0;
}
```

Output : -



```
Enter a number: 1024
Reversed Number: 4201
Process returned 0 (0x0)   execution time : 3.135 s
Press any key to continue.
```

do-while loop

C's third loop statement is the do-while loop, is an exit controlled loop i.e. it tests the conditions after having executed the statement with in loop body. This means unlike the for and while loops, a do while loop always executes at least once. The statement of the do-while loop is as follows:

```
do
{
    loop-boody ;
}while (conditional expression) ;
```

The braces { } can be discarded when the loop-boody contains a single statement. The do-while loop iterates the loop body as long as the specified condition is true while testing the condition at the end of the loop each time, rather than at the beginning, as is done by the for and the while loop.

Like while loop, do-while loop also doesn't contain the initialization and updation expression as part of loop statement. However, these expressions can be associated with do-while loop by the programmer according to required logic. Then the new form of do-while loop may looks like as:

```
Initialization expression(s);
```

```
do
```

```
{
```



```

Loop body;
Updating expression;
}while ( conditional expression(s) );

```



Example: Consider the following segment of code :

```

i=1;
do
{
printf("\n Hello World!");
++i;
} while ( i<=10);

```

where i is an integer variable declared already

i=1; is an initial expression.

i < = 10; is a conditional expression.

++i; is an update expression.

When the program control reaches at a 'do while' loop, the following events occur:

1. The loop body will be executed i.e. the print statement and the updation statement.
2. The conditional expression will be evaluated i.e. i <=10.
3. The conditional expression will evaluates to true as the value as i is 2 this time (initially=1).
4. Since the condition expression is true, the control will move back to execute the loop-body once again.

The output of the above code may looks likes as:

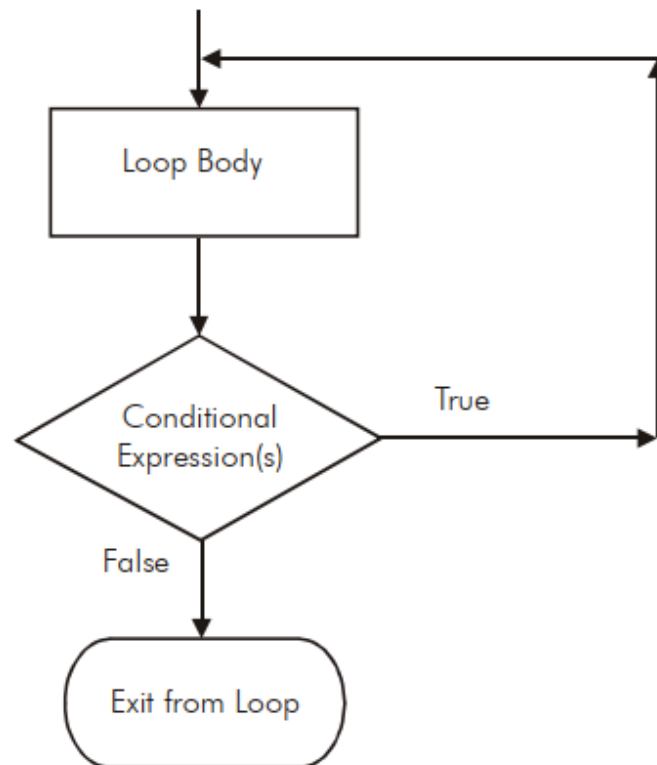
Hello World !

Hello World !

:

Hello World ! (10 times)

Figure demonstrate working of do-while loop



Program to print series of number after enter first number



Lab Exercise:

Program :-

```
#include<stdio.h>
int main(){

    int number;
    printf("Enter a number:");
    scanf("%d",&number);
    do{

        printf("Value of number is= %d\n",number);
        ++number;

    }while(number<=20);
    return 0;

}
Output:-
```

```
Enter a number:10
Value of number is= 10
Value of number is= 11
Value of number is= 12
Value of number is= 13
Value of number is= 14
Value of number is= 15
Value of number is= 16
Value of number is= 17
Value of number is= 18
Value of number is= 19
Value of number is= 20
Process returned 0 (0x0)   execution time : 21.635 s
Press any key to continue.
```

8.8 Jump and Break Statement

What if you need to get out of a loop statement before the test condition fails? The break statement can be used. Break is a statement that is used to end loops or exit from a switch (discussed later). When a break occurs within a C loop, the loop is immediately terminated without verifying the loop condition, and control is passed to the first statement following the loop. It can be used in a while loop, a do-while loop, a for loop, or a switch loop. The break statement is simply written break;

The break statement does not have any operand.

Following C code snippets illustrate use of break statement to exit from various C loops. In each situation, the loop will continue to execute as long as the current value for the integer variable x does not exceed 10. However, the computation will break out of the loop if a negative value for x is detected.

While loop

```
scanf ("%d", &x);
while (x <= 10)
{
    if (x < 0)
    {
        printf ("Negative value entered!!!\n");
        break;
    }
    scanf ("%d", &x);
}
```

do-while loop

```
do
{
```

```
scanf ("%d", &x);
if (x < 0)
{
printf ("Negative value entered");
break;
}
} while (x <= 10);
```

for loop

```
for (i = 1; x <= 10; ++i)
{
scanf ("%f", &x);
if (x < 0)
{
printf ("Negative value entered!!");
break;
}
}
```

When break is used in nested while, do-while, for or switch statements, it will cause a transfer of control out of the immediate enclosing statement, but not out of the outer surrounding statements.

Consider the following code snippet in which a while loop is nested within a for loop.

```
for (i = 0; i <= n; ++i)
{
while ((c = getchar()) != '\n')
{
if (c == '*') break;
}
}
```

8.9 goto Statement

In early programming languages, goto was a common looping idiom for unconditionally branching to one statement from another. For looping immediately, no condition is verified. The usage of goto branching is often discouraged due to the inherent complications connected with it.

C enables the goto statement to branch unconditionally from one point in the programme to another for backward compatibility. After a goto is encountered, a goto statement utilises an identifier called label to specify the statement to which branching will begin. Any valid identifier name must be followed by a colon in a label. A label is placed immediately before the statement where the control is to be transferred.

The general forms of goto and label statements are shown below:

```
goto label;
.....
.....
```

label: statements;

.....

statement;



Lab Exercise:

Program to demonstrate working of goto statement*/

```
#include<stdio.h>
```

```
int main(){
```

```
int i;
```

```
for(i=0;i<=10;i++)
```

```
{
```

```
    if(i==5)
```

```
    {
```

```
        goto STATUS;
```

```
    }
```

```
    printf("%d\n",i);
```

```
}
```

```
STATUS:
```

```
    printf("Value of i is = 5 than goto working");
```

```
return 0;
```

```
}
```

Output :-

```
0
1
2
3
4
Value of i is = 5 than goto working
Process returned 0 (0x0) execution time : 0.013 s
Press any key to continue.
```

Summary

- Most of the programs require a statement or set of statements to be executed multiple times or not to execute at all, depending on the circumstances.

- The statement by which we can control the flow of the program execution is called as control flow statement or program control statement.
- In the sequence construct, as the name implies, statements are executed sequentially i.e. one after the other.
- In selection construct, the execution of statements depends upon a condition test.
- The iteration constructs are an efficient method of handling a series of statements that must be repeated a variable number of times.
- If multiple statements are to be executed then they must be placed within a pair of braces.
- A simple if or if else construct may be placed within another if or if-else construct.
- The switch statement is another convenient tool provided by C to handle the situations in which multiple decisions to be made based on an expression that can have multiple values.
- The for loop in C is the simplest, fixed and entry controlled loop. An infinite for loop can be created by skipping the conditional expression.
- A conditional expression cannot have multiple expression like initialization and updation expression, but it may contain several conditions linked together using logical operators.
- The second type of loop, the while loop is an entry controlled loop as it tests the conditions first and if the condition is true, then only the control will enter into the loop body. An empty loop can also be configured using while statement and could be used as a time delay loop.
- C's third loop statement is the do while loop, is an exit controlled loop i.e. it tests the conditions after having executed the statement within loop body. Unlike the for and while loops, a do while loop always executes at least once.
- The break statement is used in a program to skip the particular part of program code. The jump statement continue is the complement of the break statement.

Keywords

Conditional statement: A statement that evaluates to either true or false.

Continue statement: The statement that ignores execution of further statements and forces the loop to evaluate the loop condition once again.

Default statement: An optional statement in a switch that is executed if none of the conditions evaluates to true.

Switch statement: A multi-selection statement that branches to that statement whose specified condition evaluates to true.

Control Statements: The statements that allow programmers to alter the sequential flow of execution of the program and control the flow are called control statements.

For Loop: A for loop allows execution of a statement (or a block of statements) repeatedly a number of times.

While Loop: In case the number of times a statement is to be executed is not known in advance, while loop is used.

goto statement : The goto statement is known as jump statement in C.

Self Assessment

1. What are the components of design structure of C?
 - A. Documentation
 - B. Link Section
 - C. Definition Section

-
- D. Above all
2. Every C program must have _____ function.
- A. Printf ()
 - B. Scanf ()
 - C. main ()
 - D. none of above
3. What are the Advantages of Top-Down Design?
- A. Important modules are designed first
 - B. Testing and debugging is easier and fast
 - C. Project implementation is easy
 - D. All of the above
4. How many type of conversion are there in c?
- A. 3
 - B. 2
 - C. 1
 - D. 4
5. Which conversion also called Automatic Type Conversion?
- A. Explicit Type Conversion
 - B. Implicit Type Conversion
 - C. All of above
 - D. None of the above
6. Which type of conversion is NOT accepted?
- A. From char to int
 - B. From float to char pointer
 - C. From negative int to char
 - D. From double to char
7. Type modifiers in C are
- A. Int, main
 - B. Sub program
 - C. Short, long, signed and unsigned
 - D. All of above
8. Which one is decision making statement in C ?
- A. If statement

- B. While loop
- C. Switch
- D. For loop

9. What will be output for the following code?

```
#include <stdio.h>

void main()
{
    int x = 5;
    if (true);
    printf("hello");
}
```

- A. It will display hello
- B. It will throw an error
- C. Nothing will be displayed
- D. Compiler dependent

10. Which of the following is an invalid if-else statement?

- A. if (if (a == 1)){}
- B. if (func1 (a)){}
- C. if (a){}
- D. if ((char) a){}

11. The label in goto statement is same like

- A. Case in switch statement
- B. Initialization in for loop
- C. Continuation condition in for loop
- D. All of them

12. Goto statement is also known as _____

- A. If statement
- B. Jumping statement
- C. Loop statement
- D. Above all

13. Choose a right C Statement.

- A. Loops or Repetition block executes a group of statements repeatedly.
- B. Loop is usually executed as long as a condition is met.
- C. Loops usually take advantage of Loop Counter
- D. All the above.

14. The break statement is used to exit from:

- A. a DO loop.
- B. a FOR loop.
- C. a SWITCH statement.
- D. all of above.

15. Which keyword is used to come out of a loop only for that iteration?

- A. break
- B. continue
- C. return
- D. none of the mentioned

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. C | 3. D | 4. B | 5. B |
| 6. B | 7. C | 8. A | 9. B | 10. A |
| 11. A | 12. B | 13. D | 14. D | 15. B |

Review Questions

1. Write a program using if-else statement.
2. Explain nested-if statement with example.
3. What do you mean by switch statement? How it used
4. A five-digit number is entered through the keyboard. Write a program to obtain the reversed number and to determine whether the original and reversed numbers are equal or not.
5. Write a program to check whether a triangle is valid or not, when the three angles of the triangle are entered through the keyboard. A triangle is valid if the sum of all the three angles is equal to 180 degrees.
6. Given the length and breadth of a rectangle, write a program to find whether the area of the rectangle is greater than its perimeter. For example, the area of the rectangle with length = 5 and breadth = 4 is greater than its perimeter.
7. What is the use of if-else statement?
8. Define selection in c programming.
9. Write a program in C to enter five integer values as age of five boys and calculate the average age of all the boys.
10. Write a program to calculate the area of a square. All values enter with the help of keyboard.
11. What do you mean by looping?
12. Describe for loop with the help of suitable example.
13. Differentiate while loop and do-while loop.
14. What is the advantage of break statement in while loop?

15. Write a program to find the factorial value of any number entered through the keyboard.
16. Write a program to print all the ASCII values and their equivalent characters using a while loop.
The ASCII values vary from 0 to 255.



Further Readings

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008.

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi.

Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi.

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C



Web Links

<https://www.tutorialspoint.com/index.htm>

www.webopedia.com

www.web-source.net